

LABORATORIO 00

Tu primera libreta

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

En este laboratorio creas tu primera tabla Iceberg y le escribes tres contribuyentes. Es corto a propósito. Sirve para conocer el cuaderno, saber dónde estás parado dentro del ambiente y ver que una tabla Iceberg se crea y se lee con SQL corriente.

2 · El problema, en el almacén

En el almacén de la esquina, el almacenero lleva una libreta donde anota qué entra y a qué hora. Antes de anotar algo tiene que saber dos cosas. En qué estante está parado, porque en la bodega hay varios estantes y cada uno tiene sus propias libretas, y cuál es su libreta, para no escribir en la de otro. Si no sabe eso, anota en cualquier parte.

3 · El problema en términos técnicos

Una base de datos se organiza en espacios de trabajo, que Spark llama databases. Cada sentencia se ejecuta dentro de un espacio, y si no dices cuál, se ejecuta en el que está activo. Crear una tabla en el espacio equivocado la deja donde nadie la busca. Además, una tabla común no guarda su historia. Para que la tabla sea una libreta que no borra lo anterior, hay que pedirlo al crearla.

4 · El diagrama



default (gris). Es el espacio en que parte cada cuaderno. Nadie trabaja ahí.

curso (gris). Guarda las tablas de referencia que trae el ambiente. Se leen, no se escriben.

mi_espacio (azul). Es tu espacio de trabajo. Ahí vas a crear tu tabla.

contribuyentes_lab00 (amarillo). Es tu primera tabla. `USING iceberg` hace que sea una tabla Iceberg, una libreta que guarda cada anotación como una página nueva.

3 contribuyentes (verde). Son las tres filas que escribes, y que después lees.

5 · La solución

En el almacén, el almacenero primero mira dónde está, después se va a su estante, abre una libreta nueva y escribe la primera línea. Todavía no la usa para nada, pero ya es suya.

En la tecnología, `current_database()` dice en qué espacio estás, `USE mi_espacio` te lleva a tu espacio, `CREATE TABLE ... USING iceberg` crea la tabla como tabla Iceberg, `INSERT INTO` le escribe filas y `SELECT` las lee.

6 · Los pasos

- **Paso 0.** Preguntas en qué espacio estás.
- **Paso 1.** Ves qué espacios hay y te pones en `mi_espacio`.
- **Paso 2.** Creas tu tabla Iceberg.
- **Paso 3.** Le escribes tres contribuyentes.
- **Paso 4.** Los lees.

Si quieres repetir el laboratorio desde el principio, corre este comando desde la carpeta del repositorio.
Borra la tabla del laboratorio.

```
bash
bin/reiniciar-lab.sh 00
```

Paso 0 · Entrar y mirar alrededor

Celda 0.1 · ¿Dónde estoy?

La pregunta. ¿En qué espacio de trabajo estás cuando abres el cuaderno?

Por qué ahora. Antes de crear nada conviene saber dónde va a quedar. Es la pregunta más básica, y también sirve para despertar a Spark.

En el almacén. Es mirar alrededor al entrar a la bodega para saber frente a qué estante estás parado.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor, y sirve para saber qué celda es.
- `SELECT` pide un resultado.
- `current_database()` es una función que responde el nombre del espacio en que estás. Los paréntesis vacíos indican que no recibe nada.
- `AS estoy_en` le pone nombre a la columna del resultado.

```
sql
%%sql
-- Celda 0.1
SELECT current_database() AS estoy_en
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:05:31 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
estoy_en
default
1 fila.
```

Cómo se lee. Estás en `default`, el espacio con que parte Spark. Una fila, una columna llamada `estoy_en`.

Las tres primeras líneas no son errores. Salen solo en la primera celda, cuando Spark arranca, y esta celda demora unos segundos por eso.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores, no todo lo que hace.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. No hace falta tocarlo. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:05:31` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Al final, `1 fila.` es el conteo que agrega el cuaderno debajo de cada resultado.

Paso 1 · Ver qué hay, y ponerte en tu espacio

Celda 1.1 · Los espacios del ambiente

La pregunta. ¿Qué espacios de trabajo existen en este ambiente?

Por qué ahora. Ya sabes que estás en `default`. Antes de moverte, ves adónde te puedes mover.

En el almacén. Es recorrer con la vista los estantes de la bodega y leer el nombre de cada uno.

La sentencia, parte por parte.

- `SHOW DATABASES` lista todos los espacios de trabajo que conoce el catálogo, el clavo del que cuelgan las libretas.

```
sql
%%sql
-- Celda 1.1
SHOW DATABASES
```

Lo que sale en pantalla.

```
text
namespace
curso
default
mi_espacio
3 filas.
```

Cómo se lee. Hay tres espacios. La columna se llama `namespace`, que es otro nombre para espacio de trabajo.

- `curso` guarda las tablas de referencia que vas a leer.
- `default` es el espacio con que parte Spark, y no se usa.
- `mi_espacio` es el tuyo, donde creas tus tablas.

Celda 1.2 · Ponerte en tu espacio

La pregunta. ¿Cómo te mueves a tu espacio?

Por qué ahora. La tabla que vas a crear tiene que quedar en `mi_espacio`, no en `default`.

En el almacén. Es caminar hasta tu estante. Desde ahí, toda libreta que abras es de ese estante.

La sentencia, parte por parte.

- `USE` cambia el espacio activo. Desde aquí, todo nombre de tabla sin espacio adelante se busca en el espacio que indiques.
- `mi_espacio` es el espacio al que te mueves.

```
sql
%%sql
-- Celda 1.2
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia se ejecutó. `USE` no consulta nada, te mueve, así que no hay tabla que mostrar. El cuaderno solo avisa que terminó bien.

Celda 1.3 · Comprobar el cambio

La pregunta. ¿Quedaste de verdad en `mi_espacio`?

Por qué ahora. Antes de crear la tabla conviene comprobar que el `USE` funcionó.

En el almacén. Es mirar de nuevo el cartel del estante para asegurarte de que llegaste al tuyo.

La sentencia, parte por parte. Es la misma de la celda 0.1.

- `SELECT current_database()` responde el nombre del espacio activo.
- `AS estoy_en` le pone nombre a la columna.

```
sql
%%sql
-- Celda 1.3
SELECT current_database() AS estoy_en
```

Lo que sale en pantalla.

```
text
estoy_en
mi_espacio
1 fila.
```

Cómo se lee. Ahora dice `mi_espacio`. El `USE` funcionó, y todo lo que crees de aquí en adelante queda en tu espacio.

Paso 2 · Crear tu primera tabla

Celda 2.1 · Una línea de seguridad

La pregunta. ¿Cómo te aseguras de poder repetir el laboratorio sin que falle?

Por qué ahora. Si corres el cuaderno de nuevo con la tabla ya creada, el `CREATE TABLE` falla diciendo que ese nombre ya existe. Esta línea lo evita.

En el almacén. Antes de abrir una libreta nueva con un nombre, sacas del estante la vieja que tenga ese mismo nombre, si es que hay una.

La sentencia, parte por parte.

- `DROP TABLE` borra una tabla.
- `IF EXISTS` quiere decir si existe. Si la tabla no existe, la sentencia no hace nada y tampoco se queja.
- `contribuyentes_lab00` es el nombre de la tabla. Como estás en `mi_espacio`, se busca ahí.

```
sql
%%sql
-- Celda 2.1
DROP TABLE IF EXISTS contribuyentes_lab00
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

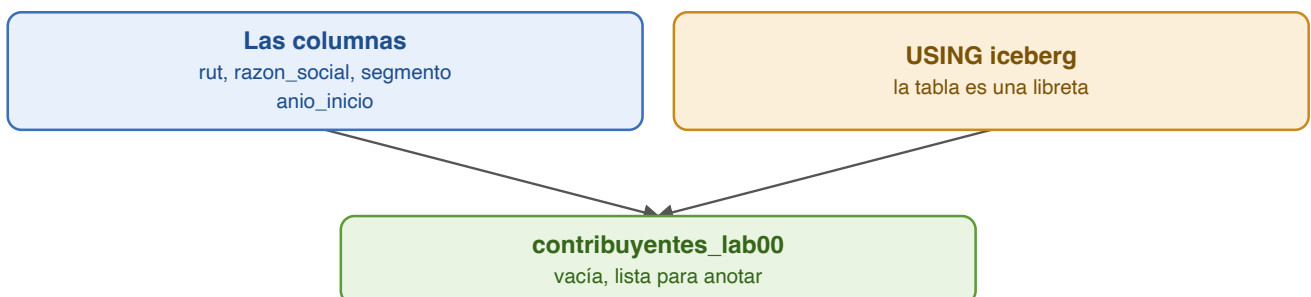
Cómo se lee. Terminó bien. La primera vez no borra nada, porque la tabla todavía no existe.

Celda 2.2 · Crear la tabla

La pregunta. ¿Cómo se crea una tabla Iceberg?

Por qué ahora. Ya estás en tu espacio y el nombre está libre.

En el almacén. Es abrir una libreta nueva y dibujar las columnas en la primera hoja, antes de anotar nada.



Las columnas (azul). Cada columna tiene un nombre y un tipo. Tres son texto y una es número entero.

USING iceberg (amarillo). Hace que la tabla sea Iceberg y no una tabla común. Por eso cada vez que escribas en ella se va a anotar una página nueva, sin borrar la anterior.

contribuyentes_lab00 (verde). La tabla queda creada y vacía.

La sentencia, parte por parte.

- `CREATE TABLE contribuyentes_lab00` crea una tabla con ese nombre en el espacio activo. Lleva `lab00` para que no choque con otras tablas del espacio.
- Entre paréntesis van las columnas, separadas por comas, cada una con su nombre y su tipo.
- `rut STRING` es una columna de texto con el RUT del contribuyente. `STRING` quiere decir texto.
- `razon_social STRING` es el nombre de la empresa, también texto.
- `segmento STRING` es el tamaño del contribuyente, MICRO, PEQUENA, MEDIANA o GRANDE. Va sin eñe, como se guarda en los datos del ambiente.
- `anio_inicio INT` es el año de inicio de actividades. `INT` quiere decir número entero.
- `USING iceberg` dice con qué formato se guarda la tabla.

```
sql
%%sql
-- Celda 2.2
CREATE TABLE contribuyentes_lab00 (
  rut          STRING,
  razon_social STRING,
  segmento     STRING,
  anio_inicio  INT
) USING iceberg
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada. Todavía no tiene filas.

Paso 3 · Escribir tres filas

Celda 3.1 · Anotar tres contribuyentes

La pregunta. ¿Cómo se escriben filas en la tabla?

Por qué ahora. La tabla existe pero está vacía. Es la libreta abierta, todavía sin nada anotado.

En el almacén. Es escribir los tres primeros renglones de la libreta. Al hacerlo queda la primera página.

La sentencia, parte por parte.

- `INSERT INTO contribuyentes_lab00` agrega filas a la tabla.
- `VALUES` introduce las filas escritas a mano. Cada fila va entre paréntesis y las filas se separan con comas.
- `'77884562-8'` es un texto, por eso va entre comillas simples. Así van el RUT, la razón social y el segmento.
- `2018` es un número, por eso va sin comillas. Es la diferencia entre `STRING` e `INT`.
- Los valores de cada fila van en el mismo orden que las columnas de la tabla, `rut`, `razon_social`, `segmento` y `anio_inicio`.

```
sql
%%sql
-- Celda 3.1
INSERT INTO contribuyentes_lab00 VALUES
  ('77884562-8', 'Araucaria Ferreteria SpA', 'PEQUENA', 2018),
  ('78800840-6', 'Copihue Maquinarias EIRL', 'MEDIANA', 2021),
  ('76171162-8', 'Huemul Alimentos EIRL', 'GRANDE', 2011)
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Las tres filas quedaron escritas. El cuaderno no las muestra porque `INSERT` no consulta nada.

Paso 4 · Leerlas

Celda 4.1 · Leer la tabla

La pregunta. ¿Qué quedó guardado en la tabla?

Por qué ahora. Escribiste tres filas. Ahora compruebas que están.

En el almacén. Es abrir la libreta y leer lo que anotaste.

La sentencia, parte por parte.

- `SELECT *` pide todas las columnas. El asterisco significa todas.
- `FROM contribuyentes_lab00` dice de qué tabla leer. Como estás en `mi_espacio`, se busca ahí.

```
sql
%%sql
-- Celda 4.1
SELECT * FROM contribuyentes_lab00
```

Lo que sale en pantalla.

```
text
rut          razon_social      segmento  anio_inicio
77884562-8   Araucaria Ferreteria SpA  PEQUENA   2018
78800840-6   Copihue Maquinarias EIRL  MEDIANA   2021
76171162-8   Huemul Alimentos EIRL    GRANDE    2011
3 filas.
```

Cómo se lee. Las tres filas que escribiste, con sus cuatro columnas. Araucaria Ferretería es PEQUENA y partió en 2018, Copihue Maquinarias es MEDIANA y partió en 2021, y Huemul Alimentos es GRANDE y partió en 2011.

Salen en el mismo orden en que las escribiste, pero eso no está garantizado. Sin `ORDER BY`, el motor puede devolver las filas en cualquier orden.

Preguntas frecuentes

¿Por qué la primera celda demora tanto?

Porque en esa celda arranca Spark. Reserva memoria, se conecta al catálogo y deja lista la sesión. Pasa una sola vez por cuaderno. Las celdas siguientes responden casi al instante.

¿Por qué `segmento` dice PEQUENA sin eñe?

Porque así están guardados los segmentos en los datos del ambiente. Se evitan los caracteres especiales en los valores que se comparan, para que un `WHERE segmento = 'PEQUENA'` no falle por cómo se escribió la eñe.

¿Qué pasa si me equivoco y escribo la tabla en `default` ?

Queda en `default`, y las celdas que la buscan en `mi_espacio` no la encuentran. Se arregla con `USE mi_espacio` y creándola de nuevo. La que quedó en `default` se borra con `DROP TABLE default.contribuyentes_lab00`.

¿Qué pasa si una celda falla?

El cuaderno muestra el mensaje de Spark. Léelo, corrige la sentencia y vuelve a ejecutar la celda. No hace falta partir de nuevo.

¿Qué pasa si corro el `INSERT` dos veces?

La tabla queda con seis filas, cada contribuyente dos veces. Una tabla Iceberg no tiene llave que impida repetir una fila. Para volver a tres, corre el laboratorio desde el principio, que empieza con el `DROP TABLE`.

¿Dónde quedó guardada la tabla?

En el almacenamiento del ambiente, en una carpeta de `mi_espacio`. Sigue ahí aunque cierres el cuaderno o apagues el ambiente.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Creaste una tabla Iceberg y le escribiste datos. Esa tabla es tuya y vive en el almacenamiento del ambiente.

En el almacén. Te pusiste frente a tu estante, abriste una libreta nueva y escribiste la primera página. Todavía no la usas para nada, pero cada vez que escribas en ella va a anotar una página nueva sin borrar la anterior.

Lo que hiciste	Con qué
Saber dónde estás	<code>SELECT current_database()</code>
Ver los espacios	<code>SHOW DATABASES</code>
Ponerte en tu espacio	<code>USE mi_espacio</code>
Crear una tabla Iceberg	<code>CREATE TABLE ... USING iceberg</code>
Escribir filas	<code>INSERT INTO ... VALUES</code>
Leerlas	<code>SELECT * FROM</code>

La diferencia con una tabla común está en dos palabras, `USING iceberg`. Todavía no se ve, pero por debajo ya está ocurriendo.